System T and Computational Power

Lecture 10

Section 1

Part I: Where We Are

What we have built so far

We have the **Simply Typed Lambda Calculus** (STLC):

- Types: $A \to B$, $A \times B$, $A + B$, $\mathbf{1}$
- Terms: $\lambda x{:}A.\, t$, application, pairs, injections, case
- **Weak normalization**: every term has a normal form
- **Curry–Howard correspondence**: types are propositions of intuitionistic propositional logic

Type	**Proposition**
$A \to B$	$A \Rightarrow B$ (implication)
$A \times B$	$A \wedge B$ (conjunction)
$A + B$	$A \vee B$ (disjunction)

A closed chapter

Every typing rule has a logical reading:

- λ-abstraction = implication introduction
- Application = implication elimination
- Pair = conjunction introduction
- Projection = conjunction elimination
- ...

Normalization = proofs can always be simplified.

At the propositional level, this is a complete picture.

But now we ask a different question...

The question for today

We know what STLC can **prove**.

What can it **compute**?

Section 2

Part II: The Computational Power of STLC

Church numerals

We can *encode* natural numbers in STLC:

$$\overline{n} \;=\; \lambda f.\, \lambda x.\, \underbrace{f\,(f\,(\cdots(f}_{n}\; x)\cdots))$$

at type $(\alpha \to \alpha) \to \alpha \to \alpha$ for any type α.

- $\overline{0} = \lambda f.\, \lambda x.\, x$
- $\overline{1} = \lambda f.\, \lambda x.\, f\, x$
- $\overline{2} = \lambda f.\, \lambda x.\, f\,(f\, x)$
- $\overline{3} = \lambda f.\, \lambda x.\, f\,(f\,(f\, x))$

The numeral $\overline{n}$ *iterates* a function f exactly n times.

Addition works

We can define addition:

$$\text{add} \;=\; \lambda m.\, \lambda n.\, \lambda f.\, \lambda x.\, m\ f\ (n\ f\ x)$$

Idea: iterate f a total of n times (via $n\ f\ x$), then m more times.

Check: $\text{add}\ \overline{2}\ \overline{3} \;\to^*\; \overline{5}$ ✓

Multiplication works

$$\text{mult} \;=\; \lambda m.\, \lambda n.\, \lambda f.\, m\,(n\; f)$$

Idea: "iterate f n times" is itself a function; now iterate *that* m times.

Check: $\text{mult}\;\overline{2}\;\overline{3} \;\to^*\; \overline{6} \quad \checkmark$

So far so good. What about exponentiation?

Exponentiation fails

Exponentiation: the idea

Observe: $\overline{n}$ is not just an iterator on *values*; it is a function on *functions*.

Given $f : \alpha \to \alpha$, the numeral $\overline{n}$ produces $f^n : \alpha \to \alpha$. So $\overline{n}$ maps $f \mapsto f^n$.

What if we **iterate that operation itself**?

$$\overline{n}(\overline{n}(f)) = \overline{n}(f^n) = (f^n)^n = f^{n^2}$$
$$\overline{n}(\overline{n}(\overline{n}(f))) = f^{n^3}$$

Apply $\overline{n}$ exactly m times $\implies f^{n^m}$.

But "apply $\overline{n}$ exactly m times" is what $\overline{m}$ does. So:

$$\overline{m}\,\overline{n}\;\; f\;\; x \;=\; f^{n^m}(x) \qquad \text{i.e.} \quad \overline{m}\,\overline{n} = \overline{n^m}$$

In the untyped λ-calculus, this works: $\overline{2}\,\overline{3} \;\to^*\; \overline{9} = \overline{3^2}$ ✓

Exponentiation: the typing problem

Write $\mathsf{Nat}_\gamma = (\gamma \to \gamma) \to \gamma \to \gamma$ for the Church numeral type at γ. We want to type-check $\overline{m}\,\overline{n}$, where $\overline{m}$ is applied to $\overline{n}$ as its first argument.

Step 1. Give $\overline{n}$ a type. It is a Church numeral, so $\overline{n} : \mathsf{Nat}_\beta = (\beta \to \beta) \to \beta \to \beta$ for some type β.

Step 2. For $\overline{m}\,\overline{n}$ to type-check, $\overline{m}$'s first argument must match the type of $\overline{n}$. We need $\overline{m} : (\alpha \to \alpha) \to \alpha \to \alpha$ with:

$$\alpha \to \alpha \;=\; (\beta \to \beta) \to \beta \to \beta$$

Exponentiation: the typing problem (cont.)

Step 3. The constraint $\alpha \to \alpha = (\beta \to \beta) \to \beta \to \beta$ looks contradictory: matching the left half gives $\alpha = \beta \to \beta$, but matching the right seems to give $\alpha = \beta$.

However, $\to$ is right-associative, so:

$$(\beta \to \beta) \to \beta \to \beta \quad = \quad (\beta \to \beta) \to (\beta \to \beta)$$

The constraint becomes $\alpha \to \alpha = (\beta \to \beta) \to (\beta \to \beta)$, giving $\alpha = \beta \to \beta$ consistently. So $\overline{m}$ is typed at $\mathsf{Nat}_{\beta \to \beta}$, while $\overline{n}$ is typed at Nat_{β}.

This type-checks and computes n^m correctly. But the two inputs require **different** Church numeral types. For addition and multiplication, both inputs share Nat_{α}. For exponentiation, there is no type making $\lambda m.\, \lambda n.\, m\ n$ work as $\mathsf{Nat} \to \mathsf{Nat} \to \mathsf{Nat}$.

The obstruction: STLC cannot say "this term works at *all* types."

A preview: what would fix this

What we would need: a **single** type $\mathsf{Nat} = \forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha$, where $\forall\alpha$ means "for all types α." Then we could instantiate m at type $\alpha \to \alpha$ and n at type α, within one polymorphic term:

$$\mathsf{exp} \;:\; \mathsf{Nat} \to \mathsf{Nat} \to \mathsf{Nat}$$

$$\mathsf{exp} \;=\; \lambda m{:}\mathsf{Nat}.\,\lambda n{:}\mathsf{Nat}.\,\Lambda\alpha.\; m\,[\alpha \to \alpha]\,(n\,[\alpha])$$

This is **System F**, which we will discuss later. Its key idea: allow $\forall$-quantifiers in types, so that terms can abstract over types. This single addition will resolve the exponentiation problem and much more.

But first: let us see what we can achieve by a different route.

Schwichtenberg's Theorem (1976)

Theorem

The functions $\mathbb{N}^k \to \mathbb{N}$ representable in **pure** STLC (function types only, no products or sums) via Church numerals are exactly the **extended polynomials**: the smallest class containing

- constants, projections
- addition, multiplication
- conditionals (if $x = 0$ then ... else ...)

and closed under composition.

No exponentiation. No primitive recursion.

STLC is logically clean but computationally weak.

The situation

System	Computational power
STLC	Extended polynomials
???	???

We need more power. Where can it come from?

Two independent diagnoses:

1. **No data**: natural numbers are encoded, there is no recursion principle.
2. **No generality**: we cannot express "this works for all types."

Section 3

Part III: System T — Adding Computational Power Directly

System T (Gödel, 1958)

Extend STLC with new **term-level** constructions (the types remain simple types):

- A base type Nat with constructors: $0 : \mathsf{Nat}$ and $\mathsf{S} : \mathsf{Nat} \to \mathsf{Nat}$
- A typed **recursor** at every type C:

$$\mathsf{R}_C \ : \ C \to (\mathsf{Nat} \to C \to C) \to \mathsf{Nat} \to C$$

Reduction rules:

$$\begin{aligned} \mathsf{R}_C\ c\ f\ 0 &\longrightarrow c \\ \mathsf{R}_C\ c\ f\ (\mathsf{S}\ n) &\longrightarrow f\ n\ (\mathsf{R}_C\ c\ f\ n) \end{aligned}$$

Note: 0, S, and R are **not** λ-abstractions or applications. They are new primitives, extending beyond the pure λ-calculus. We will return to this point.

Example: Addition in System T

$$\text{add} \;=\; \lambda m{:}\text{Nat}.\ \text{R}_{\text{Nat}}\ m\ (\lambda n{:}\text{Nat}.\ \lambda r{:}\text{Nat}.\ \text{S}\ r)$$

Here $c = m$ and $f = \lambda n.\ \lambda r.\ \text{S}\ r$, so $\text{add}\ m\ k = \text{R}_{\text{Nat}}\ m\ f\ k$.

Base case, using $\text{R}_C\ c\ f\ 0 \longrightarrow c$:

$$\text{add}\ m\ 0 \;=\; \text{R}\ m\ f\ 0 \;\longrightarrow\; m$$

Step case, using $\text{R}_C\ c\ f\ (\text{S}\ k) \longrightarrow f\ k\ (\text{R}\ c\ f\ k)$:

$$\begin{aligned}
\text{add}\ m\ (\text{S}\ k) \;=\; \text{R}\ m\ f\ (\text{S}\ k) \;&\longrightarrow\; f\ k\ (\text{R}\ m\ f\ k) \\
&=\; (\lambda n.\ \lambda r.\ \text{S}\ r)\ k\ (\text{add}\ m\ k) \\
&\longrightarrow\; \text{S}\ (\text{add}\ m\ k)
\end{aligned}$$

Concrete example: $\text{add}\ \overline{2}\ (\text{S}\ 0) \;\longrightarrow\; \text{S}\ (\text{add}\ \overline{2}\ 0) \;\longrightarrow\; \text{S}\ \overline{2} \;=\; \overline{3}$.

System T is powerful

System T can define:

- Addition, multiplication, exponentiation, factorial, Fibonacci
- GCD (Euclidean algorithm), primality testing
- All primitive recursive functions, and beyond (the Ackermann function, because C can be a function type)

Rule of thumb: if you can write it with `for`-loops (bounded iteration), System T can express it. What System T *cannot* do is unbounded search: `while`-loops that might not terminate.

Key facts (proofs later)

1. System T is **strongly normalizing**: every reduction sequence terminates. (Tait, 1967)
2. The representable functions are exactly the **provably total functions of Peano Arithmetic**. (Gödel, 1958; refined by Spector, Tait, and others)

The recursor is induction

From a Curry–Howard-inspired perspective, R's type **encodes** the induction principle. Recall: to prove P for all n, provide a base case $P(0)$ and a step $P(n) \Rightarrow P(n+1)$.

Now read R's type with C as "the property to be established":

$$\mathrm{R}_C \ : \ \underbrace{C}_{\substack{\text{base case} \\ P(0)}} \ \to \ \underbrace{(\mathrm{Nat} \to C \to C)}_{\substack{\text{given } n \text{ and a proof of } P(n), \\ \text{produce a proof of } P(n+1)}} \ \to \ \underbrace{\mathrm{Nat} \to C}_{\substack{\text{for any } n, \\ \text{a proof of } P(n)}}$$

Caveat: this is not a formal Curry–Howard correspondence. But the analogy is precise: the recursor is the *computational skeleton* of induction.

The recursor in modern practice

System T's design pattern, **define a datatype, get a recursor**, influenced the design of modern proof assistants like Lean, Coq, and Agda.

In Lean, when you define an inductive type, the system **automatically generates** a recursor:

```
inductive Nat where
  | zero : Nat
  | succ : Nat -> Nat
```

Lean generates `Nat.rec`:

```
Nat.rec : C -> (Nat -> C -> C) -> Nat -> C
```

This *is* System T's R_C. Same type, same reduction rules. One argument for the base case, one for the inductive step.

`match` is syntactic sugar for the recursor

When you write a recursive function using `match` in Lean:

```
def double : Nat -> Nat
  | Nat.zero => Nat.zero
  | Nat.succ n => Nat.succ (Nat.succ (double n))
```

this **compiles down** to an application of `Nat.rec`:

```
def double : Nat -> Nat :=
  Nat.rec Nat.zero (fun n r => Nat.succ (Nat.succ r))
```

The base case: `Nat.zero`. \ The step: given n and the recursive result $r = \texttt{double}\ n$, produce $\mathrm{S}(\mathrm{S}(r))$.

Every structural recursion on Nat in Lean is an application of `Nat.rec`: Gödel's recursor, working behind the scenes.

Every inductive type gets a recursor

The pattern generalizes beyond Nat. Define binary trees:

```
inductive Tree (A : Type) where
  | leaf : Tree A
  | node : Tree A -> A -> Tree A -> Tree A
```

Lean generates `Tree.rec`:

```
Tree.rec : C -> (C -> A -> C -> C) -> Tree A -> C
```

One argument per constructor. The recursor **is** the elimination principle for the type.

Read: to produce a C from a tree, provide:

- What to return for a `leaf` (one argument of type C)
- How to combine left result, node value, and right result ($C \to A \to C \to C$)

Example: flattening a tree to a list

Suppose we want to flatten a tree into a list by in-order traversal.

Using `match`:

```
def flatten : Tree A -> List A
  | Tree.leaf => []
  | Tree.node l x r => flatten l ++ [x] ++ flatten r
```

This desugars to `Tree.rec`:

```
def flatten : Tree A -> List A :=
  Tree.rec [] (fun fl x fr => fl ++ [x] ++ fr)
```

The recursor does the structural recursion for us: `fl` and `fr` are the **already-computed** results of flattening the left and right subtrees. We just say how to combine them.

What the recursor gives you for free

To use a recursor, you supply one function per **constructor**:

For each constructor...	**The recursor ensures...**
you give a base value	every case is covered
you give a combining function	recursive calls target only subterms
(nothing else needed)	**termination** is automatic

Lean's termination checker accepts functions defined by structural recursion without further proof: they are applications of the recursor, which is **total by definition**.

Arbitrary recursion (`while` loops, general fixpoints) can diverge. The recursor *cannot*. Structure your recursion around the data, and termination comes for free.

System T's legacy

The **computational backbone** of modern proof assistants (define data inductively, eliminate it by recursion, guarantee totality) is System T's contribution.

System T idea	**Modern incarnation**
Nat with 0, S	`inductive Nat where ...`
R_C at every type	auto-generated eliminators
structural recursion	pattern matching / `match`
totality by construction	termination checker

Every `match` in Lean, every `Fixpoint` in Coq, every pattern match in Agda descends from Gödel's recursor.

What can System T compute?

For STLC, we asked: what functions $\mathbb{N} \to \mathbb{N}$ are representable? Answer: extended polynomials (Schwichtenberg).

We should ask the same question about System T. It can obviously compute more, since we have real recursion now. But how much more?

We will need a concept: **total functions** and the question of which total functions a formal system can "certify" as total.

Total vs. partial functions

A computable function $f : \mathbb{N} \to \mathbb{N}$ is **total** if it halts on every input.

Examples of total functions: addition, multiplication, the Fibonacci sequence, sorting algorithms, the Ackermann function.

Examples of partial (non-total) functions: a program that searches for a counterexample to Goldbach's conjecture and returns it (we don't know if this halts on all inputs). Or simply: `while true do skip`.

Every System T function is total (by strong normalization). But not every total computable function is in System T. **Which** total functions does System T capture?

To answer this, we need a way to *certify* that a function is total. This leads us to **Peano Arithmetic**.

Background: Peano Arithmetic (PA)

Peano Arithmetic is the standard formal system for reasoning about natural numbers. Its language has 0, successor S, $+$, $\times$, and first-order logic ($\forall$, $\exists$, $\wedge$, $\vee$, $\neg$, $\rightarrow$).

Its key axioms:

- $\forall x.\ S(x) \neq 0$ (zero is not a successor)
- $\forall x\, \forall y.\ S(x) = S(y) \rightarrow x = y$ (successor is injective)
- Defining equations for $+$ and $\times$ ($x + 0 = x$, etc.)
- **Induction schema**: for every formula $\varphi(n)$,

$$\varphi(0) \ \wedge\ \forall n. (\varphi(n) \rightarrow \varphi(n+1)) \ \implies\ \forall n. \varphi(n)$$

To say "φ is **provable in PA**" means there exists a finite formal derivation of φ from these axioms using the rules of first-order logic.

What counts as a formal derivation?

A formal derivation in PA is a finite sequence of steps, each justified by a rule of **first-order logic** together with **PA's axioms**:

Logical rules	**What it does**
Modus ponens	from φ and $\varphi \to \psi$, conclude ψ
$\forall$-introduction	from a proof of $\varphi(x)$ for arbitrary x, conclude $\forall x.\, \varphi(x$
$\forall$-elimination	from $\forall x.\, \varphi(x)$, conclude $\varphi(t)$ for any term t
PA axiom schema	
Induction	from $\varphi(0)$ and $\forall n.\, (\varphi(n) \to \varphi(n+1))$, conclude $\forall n.\, \varphi$

"Provable in PA" means: starting from PA's axioms and applying these rules finitely many times, you can reach φ. The derivation is a **finite, mechanically checkable** certificate.

The problem: totality is undecidable

We cannot decide in general whether a program halts on all inputs. So "all total computable functions" is an intractable class; we cannot even enumerate it.

But we can ask a more refined question. Fix a formal system like **Peano Arithmetic** (PA). A function f is **provably total in PA** if PA can prove:

$$\forall x \, \exists! \, y \;\; f(x) = y$$

Example. Let $\text{isprime}(x) = 1$ if x is prime, 0 otherwise. To say isprime is provably total in PA means PA proves:

$$\forall x \, \exists! \, y \;\; \text{isprime}(x) = y$$

This is not deep: PA can express "x has no divisor between 2 and $x - 1$," and the bounded search over divisors visibly terminates. The point is that the totality claim is a *theorem of PA*, not a metatheoretic observation.

Provably total in PA

PA can prove these functions always halt:

- Arithmetic: addition, multiplication, exponentiation, factorial
- Number theory: GCD, primality testing, computing the n-th prime
- Algorithms: sorting, searching, graph traversal
- Approximation: computing the first n digits of π (as a function $\mathbb{N} \to \mathbb{N}$)
- Essentially any algorithm in a standard textbook

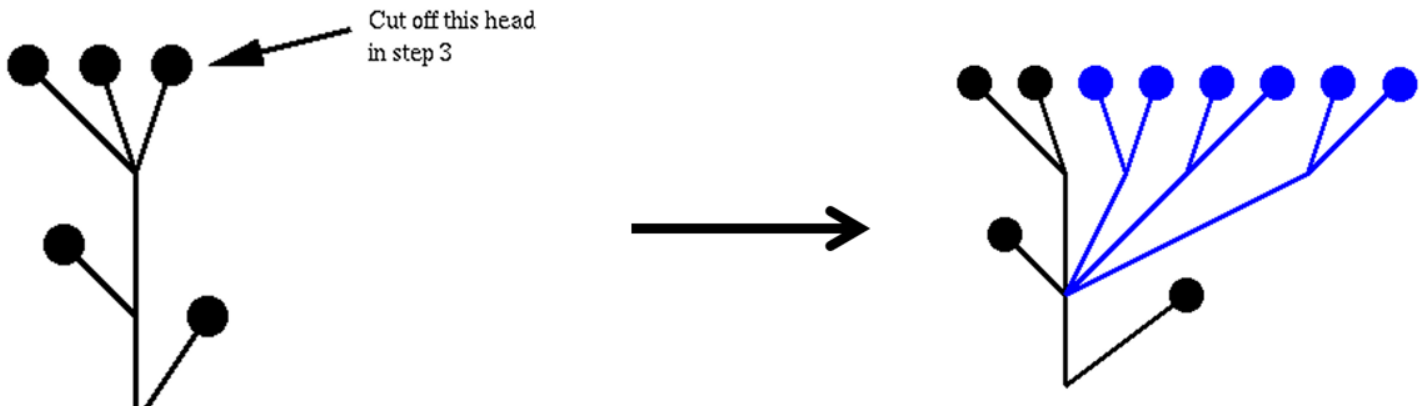
PA is very strong. If a working mathematician proves a function terminates, that proof almost certainly formalizes in PA.

Beyond PA: the Hydra game

Escaping PA requires exotic examples. Here is the most visual one.

The Hydra game (Kirby–Paris, 1982): a hydra is a finite rooted tree. At each step, you chop off a head (a leaf). The hydra retaliates:

Regrowth rule. If the head's parent is the root, nothing regrows. Otherwise, let p be the parent and g the grandparent. The subtree rooted at p (minus the removed head) is copied n times and attached to g, where n is the step number.



The Hydra game: why it matters

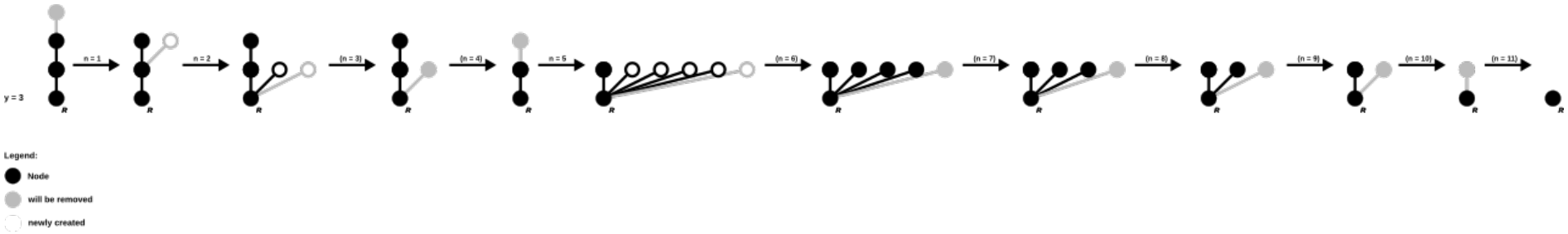
Theorem: no matter what strategy you use, the hydra always dies. But the number of steps can grow unimaginably fast.

The function $f(n) =$ "steps to kill hydra n" (fixing a strategy and encoding hydras as natural numbers) is total and computable, but PA **cannot prove it is total**. Its totality requires transfinite induction up to ε_0, exactly where PA's strength runs out.

Other examples: Goodstein sequences, Paris–Harrington. Such examples are rare, which speaks to how strong PA is.

The Hydra game: a complete example

A hydra of height 3 dying in 11 steps:



Even this small tree spawns many new heads before it is finally killed. For larger hydras, the number of steps dwarfs any number you can write down.

The characterization theorem

Theorem (Gödel, 1958; converse by Kreisel, Tait, and others)

A function $f : \mathbb{N} \to \mathbb{N}$ is representable in System T if and only if it is provably total in PA.

System T and PA carve out **exactly the same class** of functions, even though they look nothing alike: one is a type system with a recursor, the other is a first-order logical theory with induction axioms. This is evidence that the class is *natural*, not an artifact of either formalism.

Gödel's Dialectica interpretation

How did Gödel prove this? Via the **Dialectica interpretation**: a systematic translation of PA proofs into System T programs.

The key idea: when PA proves $\forall x\, \exists y\, \varphi(x, y)$, the proof implicitly contains a *method* for computing y from x. The Dialectica interpretation extracts this method as a System T term.

Concretely, the translation replaces quantifiers with functionals:

$\exists y\, \varphi(y)$	$\rightsquigarrow$	a term t of type Nat (the witness)
$\forall x\, \varphi(x)$	$\rightsquigarrow$	a function $f : \mathrm{Nat} \to \ldots$ (works for any input)
$\forall x\, \exists y\, \varphi(x, y)$	$\rightsquigarrow$	a function $f : \mathrm{Nat} \to \mathrm{Nat}$ (computes y from x)

Quantifier alternation becomes functional dependency: the computational content of the proof is a **program**.

Why did Gödel build this?

Gödel's motivation was a fundamental problem in the foundations of mathematics.

The problem: Gödel's own second incompleteness theorem (1931) shows PA cannot prove its own consistency. Any consistency proof must use methods *beyond* PA.

Gödel's idea: prove PA's consistency using **higher-type functionals** that are "evidently meaningful" even to someone who distrusts abstract set theory. A function $f : \mathrm{Nat} \to \mathrm{Nat}$ is concrete. A functional $F : (\mathrm{Nat} \to \mathrm{Nat}) \to \mathrm{Nat}$ is more abstract, but still constructive: it takes a function and returns a number.

The strategy: define System T, then use the Dialectica interpretation to show: *if System T's functionals are well-defined, then PA is consistent*.

What does "well-defined" mean here?

"System T's functionals are well-defined" means: **every System T term normalizes**. The recursor at every type actually computes a value and never diverges.

Why does this imply consistency of PA?

1. Suppose PA is inconsistent. Then PA proves $\perp$.
2. The Dialectica interpretation translates this proof into a System T term of **empty type**.
3. But if every System T term normalizes, it must reduce to a value. The empty type has **no constructors**, so no normal form exists.
4. Contradiction. So PA must be consistent.

The circularity question

The Dialectica reduces PA's consistency to normalization of System T. But by the incompleteness theorem, proving normalization **must** use principles beyond PA. There is no way around this.

So have we accomplished anything? Gödel's answer: **yes**, because the principles needed are *logically* stronger but *philosophically* simpler.

Gödel's claim: the recursor R c f n decreases n by one at each step, so it reaches 0 and stops. This is a concrete observation about a computation, not an abstract logical deduction. You can *see* it terminates.

The finitistic debate

The subtlety: at type $(\mathrm{Nat} \to \mathrm{Nat}) \to \mathrm{Nat}$, each step of the recursion produces a *functional*, i.e. something that takes an infinite object (a function on all natural numbers) as input and returns a number.

The recursor terminates in n steps, but each step involves manipulating something a finitist might not accept as a legitimate mathematical object: "I cannot inspect or verify an infinite object; how can I be sure operations on such things are well-defined?"

Gödel's position: these higher-type functionals, while abstract, are still "constructively evident" because you have a concrete recipe for computing them. Critics (including Tait) questioned whether this really counts as finitistic in Hilbert's original sense.